

Lecture 2

VLSI Design Flow overview

Prof Peter YK Cheung

Department of Electrical & Electronic Engineering

URL: <https://elec70142-digital-vlsi-design.github.io/course-website/>

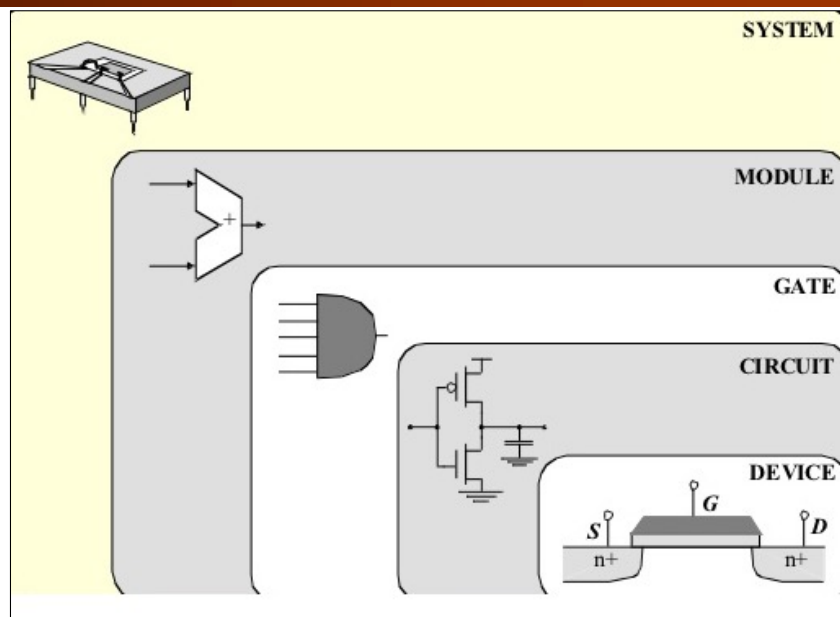
E-mail: p.cheung@imperial.ac.uk



In this lecture, we examine in some details how to go from RTL description of a digital circuit all the way to layout. You will learn the various information that are needed to complete this process.

This lecture is closely linked to Lab 1 of this module.

Levels of Abstraction in Digital Design



Digital design, unlike analogue IC design, faces two main challenges: handling complexity involved, and how to verify that such a complex design works correctly. To handle the complexity involved, a designer needs to perform the design task at various level of abstract.

When designing a VLSI chip with millions to billions transistor, a chip architect has to start at the system level (so called System-on-Chip SoC design). The SoC is divided into self-contained modules. For example, in the SoC chip in L1S21, the chip is divided into modules show in rectangle blocks.

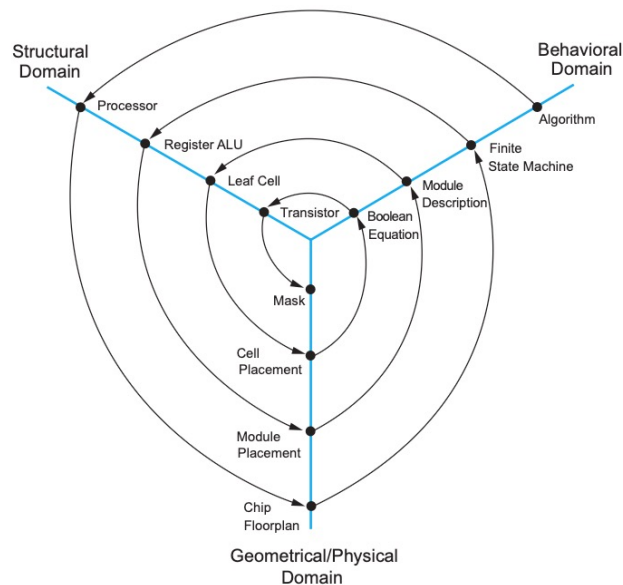
Each module is then converted into smaller digital building block, such as gates or standard cells.

The next level down is the circuit level, where each gate is made up of transistors. In our case, PMOS and NMOS.

Finally each transistor is made up of layers of materials that form the device itself.

For this module, in order that complexity can be handled, we will be designing only down to the gate or standard cell level of abstraction, not lower.

Levels of Abstraction in Digital Design



Source: Gajski 1983 & Kang 2003

This is known as the Y-chart originally introduced by Gajski in 1983. It provides an alternative view on design partitioning using three domains: behavioural, structural and geometrical or physical.

Behavioural domain has the highest level of abstraction. It describes the designer's intent without specifying exactly how the intention is achieved.

The structural domain covers the next few levels from modules down to transistors. It describes the interconnection of gates and modules, or even transistors.

The physical domain includes physical locations of gates and cells and placement of cells and blocks and the mask layers.

EDA (Electronics Design Automation) tools are used to transform or map descriptions from behavioural to structural, and finally to geometrical domains. Further tools are then used to go up the hierarchy, to verify that the final physical design matches the behavioural description. This is the verification process.

```

graph TD
    A[Design Specification] --> B[Behavioral Description]
    B --> C[RTL Description (HDL)]
    C --> D[Functional Verification and Testing]
    D --> E[Logic Synthesis/  
Timing Verification]
    E --> F[Gate-Level Netlist]
    F --> G[Logical Verification and Testing]
    G --> H[Floor Planning  
Automatic Place and Route]
    H --> I[Physical Layout]
    I --> J[Layout Verification]
    J --> K[Implementation]
    
    %% Feedback loops
    C --> C
    D --> C
    E --> C
    G --> A
    H --> C
    J --> I
  
```

The flowchart illustrates the IC design process, starting with Design Specification, followed by Behavioral Description, RTL Description (HDL), Functional Verification and Testing, Logic Synthesis/Timing Verification, Gate-Level Netlist, Logical Verification and Testing, Floor Planning Automatic Place and Route, Physical Layout, Layout Verification, and finally Implementation. The process includes feedback loops for verification and testing at various stages.

Design entry. Using a **hardware description language (HDL)** or schematic entry.

System partitioning. Divide a large system into ASIC-sized pieces.

Pre-layout simulation. Check to see if the design functions correctly.

Floorplanning. Arrange the blocks of the netlist on the chip.

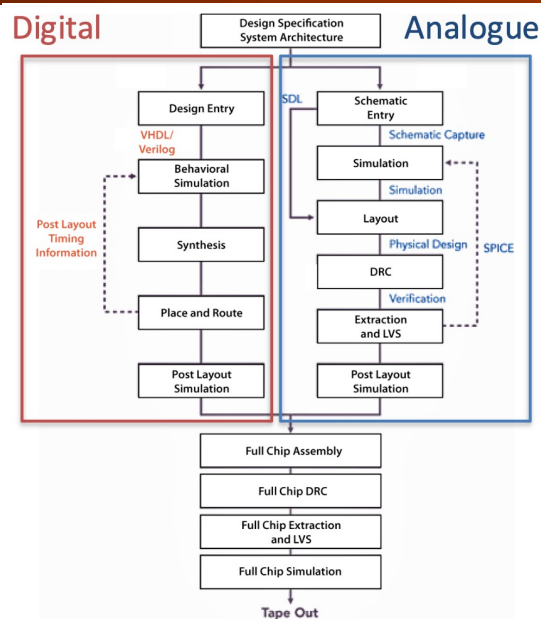
Placement. Decide the locations of cells in a block.

Routing. Make the connections between cells and blocks.

Extraction. Determine the resistance and capacitance of the interconnect.

Postlayout simulation. Check to see the design still works with the added loads of the interconnect.

Mixed signal VLSI Design Flow



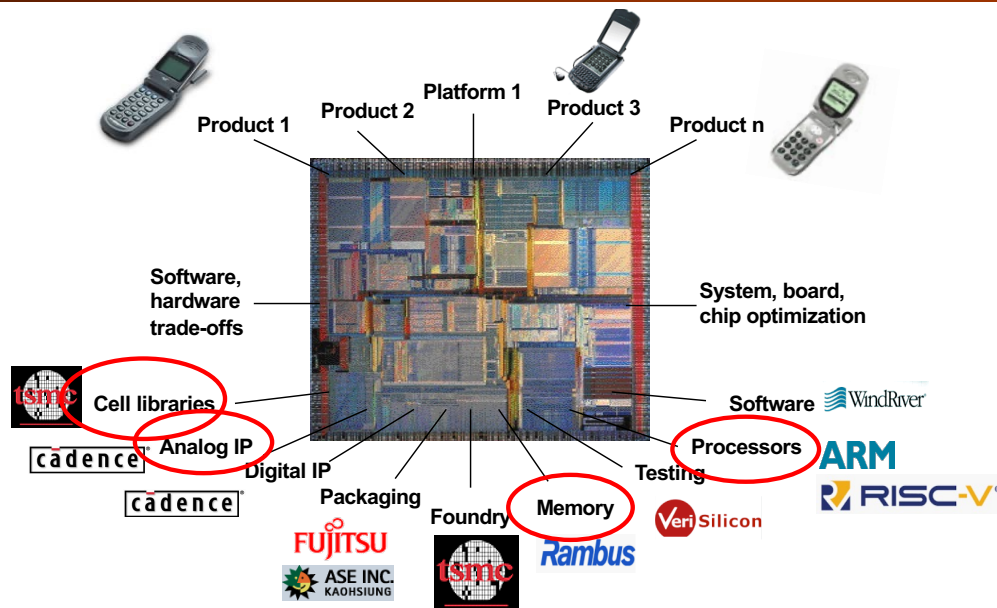
Source: Notes from T Constandinou

This is another view of how a high-level specification of a design is mapped to physical layout. With mixed signal design, much effort is put into the circuit design and simulation stage, particular for analogue part of the chip.

As can be seen in this slide, the design flow from specification to layout is different for digital and for analogue. With analogue IC design starts with schematic capture and relies heavily on circuit level simulation to verify that the design meets the specification. Layout stage is also different. It usually involve “polygon” editing. Each transistor and each interconnection must be carefully considered and handcrafted. Fortunately, the scale of a typical analogue circuit is sufficiently small that such manual design method is possible and even desirable.

For digital circuits, the design flow is totally different. The issue with digital circuits are relating to managing complexity and optimisation of area and performance. The difficulties relates to the scale of the circuit, the power and clock distributions and the timing of circuits to meet the specification.

IP based VLSI Design



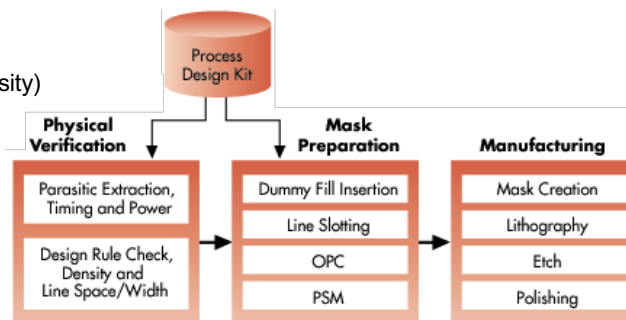
Another approach to handle the complexity of modern VLSI chips is to build a chip using intellectual property (IP) blocks designed by others. Here is an example where different companies provide the various IP modules that are fully tested down to the layout.

This approach has many advantages. It reduces the need for in-house expertise in a large variety of subjects. It further reduces both the time for designing a complex VLSI chip and reduces the risk involved.

Shown here is a complex VLSI chip using different IPs from various sources. For this module, we use exclusive TSMC's own IPs: their standard cell library for the 65nm LP process and their memory compiler for SRAM. Further, our own Imperial College team provides two IP blocks: a phase-locked loop (PLL) for clock generation and a RISC-V 32I core. The PLL was designed by Peter Shuanghua Liu. The RISC-V 32I core was designed by Luca Ridolfi. Both circuits have been verified in silicon on the 2026 tapeout.

What is a Process Design Kit (PDK)

- ◆ Link between chip maker (foundry) and EDA tool supplier (TSMC → Synopsys)
- ◆ PDK is for a specific process (we use TSMC 65nm low power process)
- ◆ PDK consists for set of files in standardised format for different design steps:
 - Primitive devices (e.g. transistors, capacitor etc.)
 - Device parameters
 - Device symbol
 - Technology related information
 - Layers and layer names
 - Appearance (colour)
 - Constraints (e.g. current density)
 - Verification information
 - Design rule checking (DRC)
 - Layout vs Schematic (LVS)
 - Device extraction
 - Simulation models
 - Standard cell libraries
 - Documentation



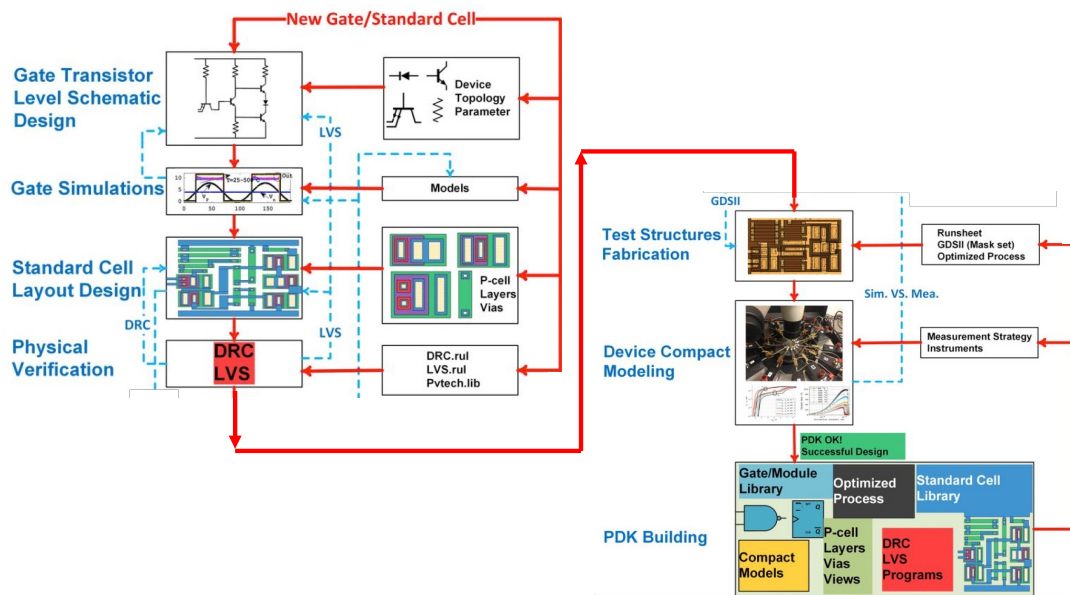
Process Design Kit (PDK) is provided by the Foundry or chip maker for specific EDA tool platform. In our case, we use TSMC 65nm PDK for Synopsys. (Every student must sign a non-disclosure agreement before accessing the PDF.)

The PDK consists of a many files divided into different categories, and includes a set of fully tested and characterised standard cells.

The files can be roughly divided into the categories shown above. These are used for:

- Full-custom design using individual **primitive devices** such as PMOS and CMOS transistors, capacitors, diodes etc.
- Information relating to the **process technology** such as different mask layers, what colour they appears in the EDA tools and some properties such as current density in metal etc.
- Information necessary for **verifying the chip** works properly. These include design rules (e.g. minimum width of a layer, minimum separation between layouts etc.); extraction rules – how layers together form a device or a component.
- **Simulation model** – for analogue (SPICE-like) and digital simulators.
- **Standard cell library** – facilitate technology mapping, timing optimization and place-and-route optimization.
- **Documentation** and design guidelines.

PDK Build Process



Source: Gupta

PYKC 30 Sep 2026

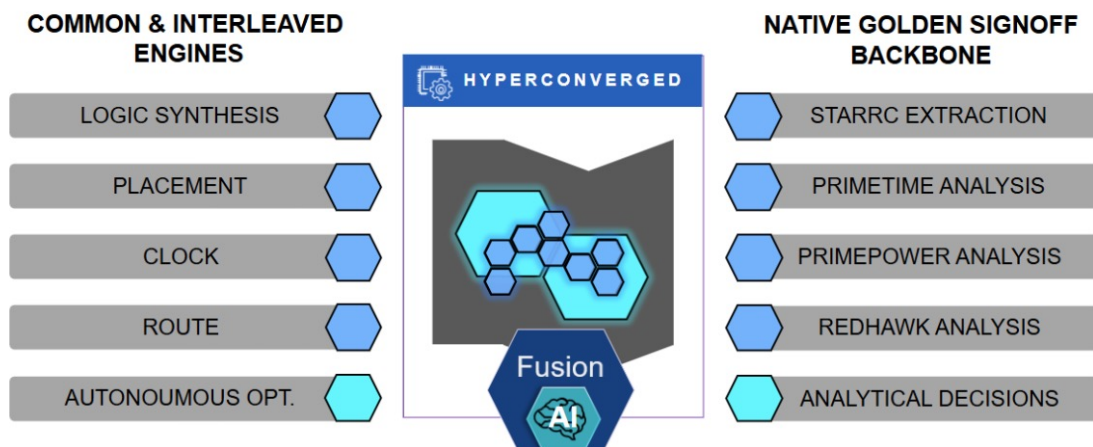
EE4 – Digital VLSI Design

Lecture 2 Slide 8

This slide explains how one would build your own PDK or your own cell library. However, we are not going to do this in our module. Firstly, we do not need to do this because TSMC provides us with a fully verified PDK. Secondly, this process is outside the scope of the module.

However, if you work for a Foundry like TSMC or UMC, this process is vital.

Synopsys Fusion Compiler



- ◆ Unified data model operating in a coherent environment.
- ◆ Avoid translation error.
- ◆ Constraints are consistent throughout.
- ◆ Speed up the learning curve of engineers.

source: Fusion Compiler User Guide

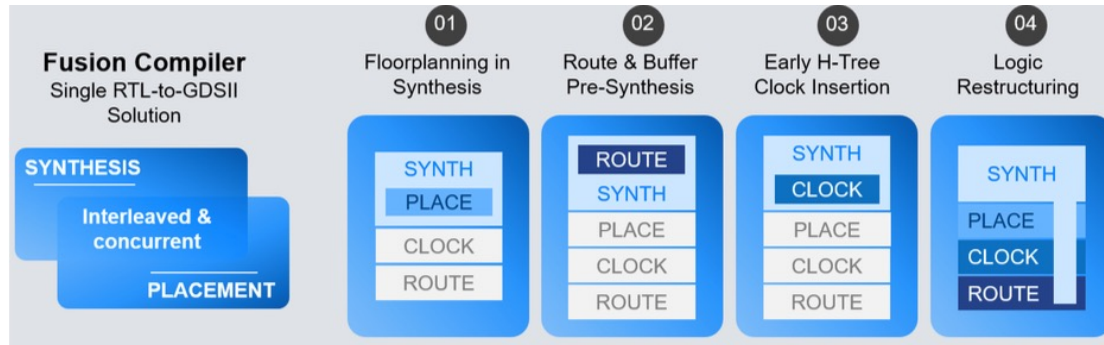
Designing a VLSI chip involve many loosely connect steps:

1. Logic Synthesis
2. Floorplanning
3. Placement
4. Clock tree synthesis
5. Routing
6. Verification and signoff

Fusion Compiler is designed to avoid these problems with a unified design flow:

- Late discovery of timing and power issues
- Large discrepancy between early estimates and final layout performances
- Take a long time to iterate through many steps before tapeout

Fusion Compiler Shell Engines



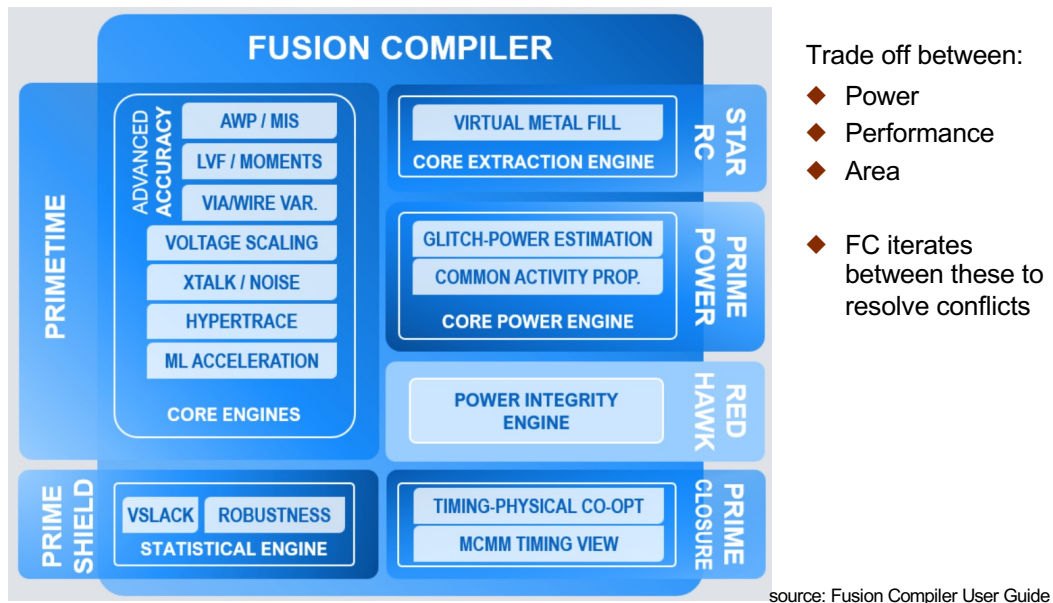
source: Fusion Compiler User Guide

Fusion Compiler (FC) uses its own shell that perform all the different stages of a complex VLSI design as shown above.

FC uses a unified data model (NDM) to represent:

- 1) Logical structures (netlists & hierarchy);
- 2) Physical information (placement and routing locations);
- 3) Timing (delay, slack, constraints);
- 4) Power intent (power domain and power network).

Power Performance Area Optimization



PYKC 30 Sep 2026

EE4 – Digital VLSI Design

Lecture 2 Slide 11

With new data model (NDM), FC can perform power-performance-area optimization efficiently.

This notes is taken from FC User Guide*:

At the heart of implementation is PPA optimization:

- Performance: Can the design meet its target frequency across all modes and corners?
- Power: Does it stay within thermal and energy budgets?
- Area: Is it cost-effective to manufacture?

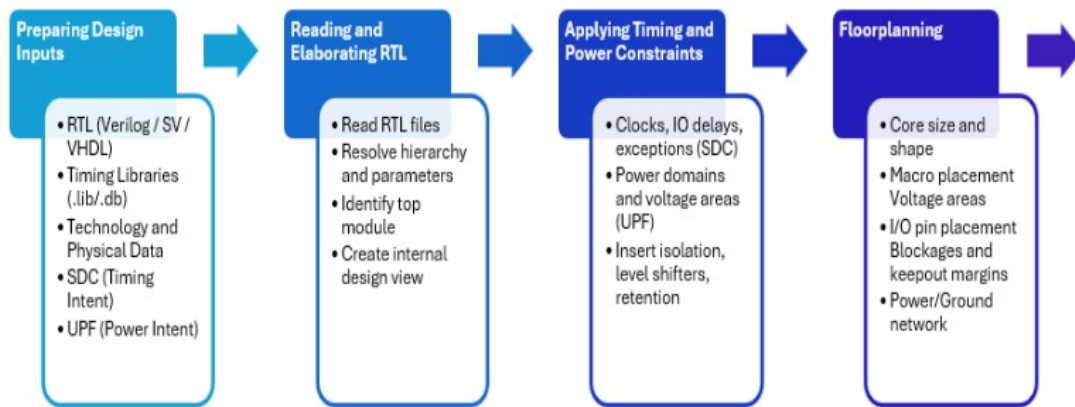
These goals are inherently conflicting. For example:

- Higher performance often requires more buffering and stronger cells, increasing power and area
- Lower power might require additional clock gating and logic restructuring, affecting timing
- Smaller area can increase congestion, harming routability and performance.

Fusion Compiler continuously evaluates these trade-offs throughout the flow, instead of postponing them until final signoff.

* https://spdocs.synopsys.com/dow_retrieve/latest/dg/fcolh/pdf/fcug.pdf

RTL to GDSII Design Flow in FC (1 of 2)

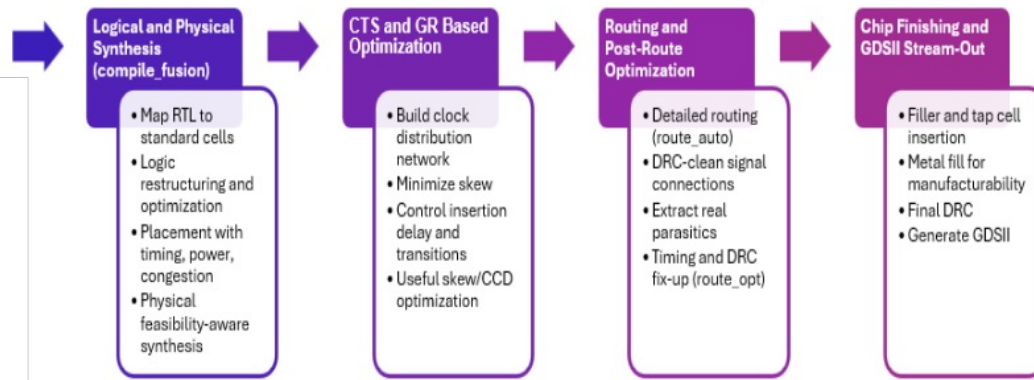


source: Fusion Compiler User Guide

This and the next slide shows the full lifecycle at each stage of Fusion Compiler doing its thing. Understanding these steps is important. Fusion Compiler is mostly driven by running commands and scripts. It relies on the HDL source code to specify the design, constraints files to provide detail information about the design intentions, and other commands to provide constraints such as aspect ratios of blocks, power and timing requirements. Knowing these individual steps will allow you to tune such constraints to steer FC to yield the optimised design in silicon.

You will be testing the first five stages of this design flow in Lab 1 using a simple circuits from your 2nd year module on circuits and systems (if you are EEE students), or on instruction architecture (if you are EIE students).

RTL to GDSII Design Flow in FC (2 of 2)

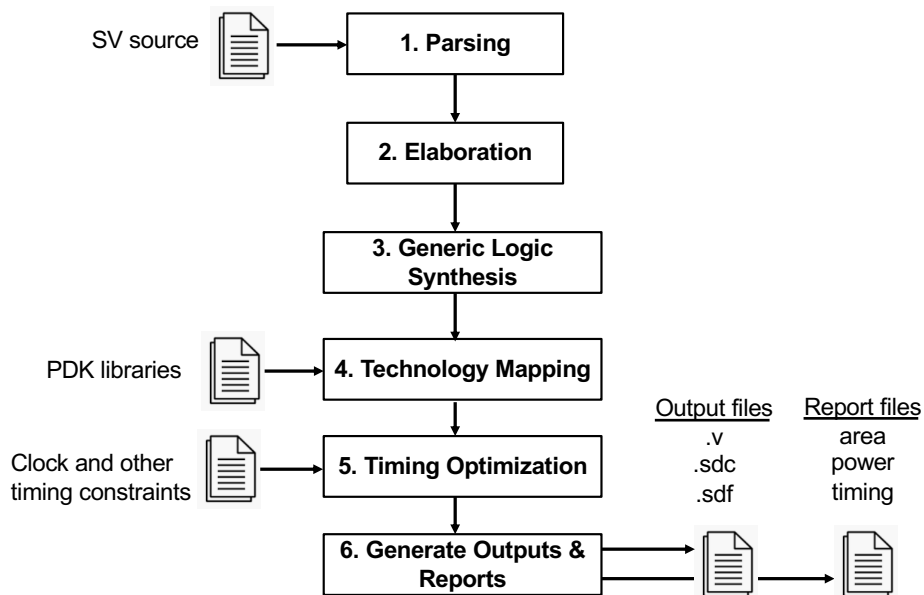


source: Fusion Compiler User Guide

These four stages drive the design towards the final layout and produced the GDSII formatted design file. In addition to the design flow shown in these two slides, you will also be using another set of tools that helps the backend process. This includes verification tools such as VCS (Verilog Compiled Simulator), IC Validator for signoff design-rule-check (DRC) and layout-vs-schematic (LVS), and TestMAX tools to make your design easy to test after manufacture.

These topics will be cover at later lectures.

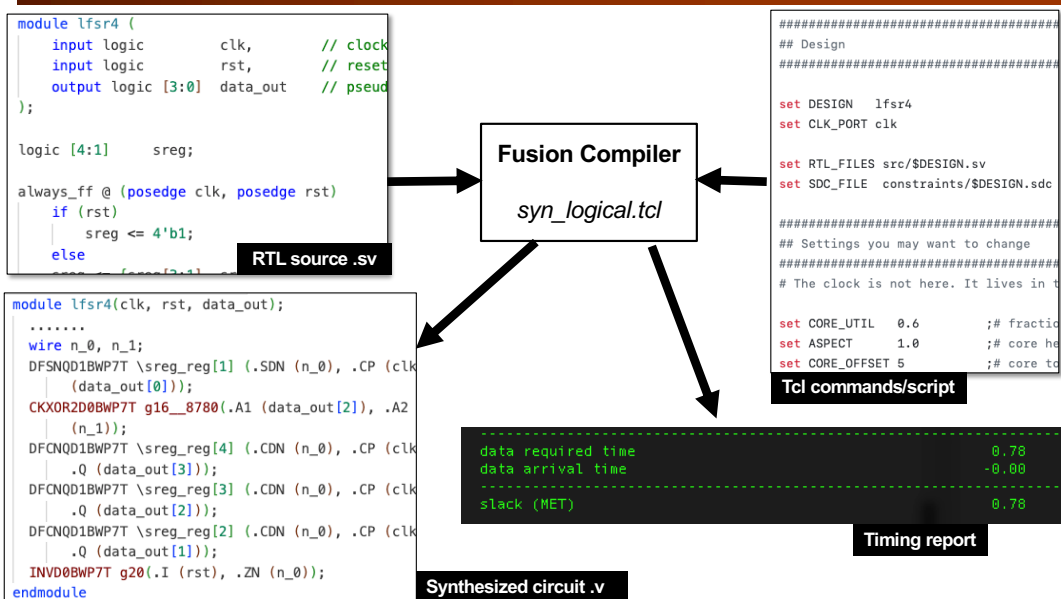
Lab 1 > Task 2: RTL to Standard Cell Synthesis



Fusion Compiler (FC) consists of many separate tools to compile HDL code to silicon. The first phase is RTL to **standard cell synthesis**, which be explored in Lab 1. This stage of the design flow involves the following steps:

- 1. Parsing** – Check for syntax and other basic errors in SystemVerilog sources.
- 2. Elaboration** – Expand modules and submodules, resolves parameters, process generate statements and check that all signals are connected.
- 3. Generic logic synthesis** - Partition RTL specification into control and dataflow graphs (CDFG) and perform initial synthesis. Turn the nodes of graphs into generic gates (e.g. NAND3 or DFF etc.) no matter what the process technology is to be used. The output of this process is a netlist of generic gates.
- 4. Technology mapping** – Map generic gates to TSMC standard cells. It also choose the correct variant of cell based on the timing, area and power constraints. It also insert tie cells (e.g. tie-hi or tie-lo) and clock gating cells as needed.
- 5. Timing optimization** – Analyse timing and use the timing constraints (e.g. clock period), optimize the standard cells for the required timing requirements (i.e. setup and hold times). Also choose the drive strength of cells for the required performance. Buffers may be inserted.
- 6. Output and report generation** – Write synthesised outputs including netlist in form of Verilog sources (.v), timing constraints (.sdc) and delay information (.sdf). Also write reports on important parameters such as area, power and delay.

Synthesize LFSR4



Fusion Compiler takes RTL source code (SystemVerilog in our case). It also needs lots of FC specific commands. The command syntax is that of **Tcl** (pronounced as tickle). You can enter these commands one at a time or source them from a script file (.tcl).

The result of this stage of Fusion Compiler is a synthesized netlist in Verilog format (not SystemVerilog). It also produces timing constraints file, delay file and various reports (e.g. area, timing, power etc.), both in log files and also as output on your terminal screen.

Synthesis Steps (*setup.tcl*) (1)

```
#####
## Design
#####

set DESIGN lfsr4
set CLK_PORT clk

set RTL_FILES src/$DESIGN
set SDC_FILE constrain#####

#####
## Settings you may want to change
#####

# The clock is not here. It lives i#####

set CORE_UTIL 0.6
set ASPECT 1.0
set CORE_OFFSET 5

set RING_WIDTH 1.0
set RING_SPACING 0.5
# Centres the ring set
# (CORE_OFFSET - (2 *
set RING_OFFSET 1.25

# Power rails on the l
set RAIL_LAYER M1

# The cell rails are c
# rather than a crossi
set RING_H_LAYER M3
set RING_V_LAYER M2#####

#####
## Directories and files
#####

set WORK_DIR work
set OUT_DIR outputs/$FLOW
set RPT_DIR reports/$FLOW
set SYN_LIB $WORK_DIR/${DESIGN}_syn.dlib
set PNR_LIB $WORK_DIR/${DESIGN}_${FLOW}.dlib

set SYNTH_V $OUT_DIR/${DESIGN}_synth.v
set SYNTH_SDC $OUT_DIR/${DESIGN}_synth.sdc
set LAYOUT_V $OUT_DIR/${DESIGN}_layout.v
```

Almost all EDA tools are driven by scripts written in tcl syntax. Therefore, it is worth you learning Tcl to improve your productivity and equip you for your future employment prospect.

See Lab 1 > Task 2 > Step 1 for links to **Tcl learning resources**.

You run the synthesis step inside the Fusion Compiler shell (*fc_shell*). You may use either individual FC commands or source a series of commands from a Tcl script. In our case, the scripts are ***setup.tcl*** and ***syn_logical.tcl***.

Running Fusion Compiler (FC) for logic synthesis involves many distinct steps.

Before synthesis can start, the script needs to:

- ❖ Set environment variable to say what files to use, folders containing files, name of clock signals, and other parameters.
- ❖ Set up database about standard cells and other useful information for the process using the PDK provided.
- ❖ Perform elaboration step and basic syntax checking.
- ❖ Inform FC about the clock signal and various clock constraints so that timing optimisation can be performed during synthesis.

Synthesis Steps (*syn_logical.tcl*) (2)

◆ Key inputs required for logic synthesis:

1. RTL source code (.sv)
2. Timing libraries (.lib or .db)
3. Technology files (.tf), parasitic models (.tlup) and physical libraries (.ndm)
4. Timing constraints (.sdc)
5. Power constraints (.upf) – not used in Lab 1

◆ The key steps in performing logic synthesis are:

1. Read and parse RTL
2. Elaborate the RTL design, check all hierarchy, link to library cells and submodules
3. Apply timing (and power) constraints
4. Perform logic synthesis (command: `compile_logical`)
5. Write report and export the design

This slide shows the input data required for logic synthesis. It also lists the steps performed in taking a RTL source code to produce a synthesis logic specification.

Note that these steps are performed using various FC shell commands which are stored in an integrated script: *syn_logical.tcl*.

In Lab 1, Task 2, you are led step by step through these commands so that you can see what FC would report on the console screen.

Output of logic synthesis step

```
module lfsr4 (
    input logic      clk,          // clock
    input logic      rst,          // reset
    output logic [3:0] data_out    // pseudorandom
);

logic [4:1] sreg;

always_ff @ (posedge clk, posedge rst)
    if (rst)
        sreg <= 4'b1;
    else
        sreg <= {sreg[3:1], sreg[3] ^ sreg[4]};

assign data_out = sreg;
endmodule
```

RTL source
in SystemVerilog

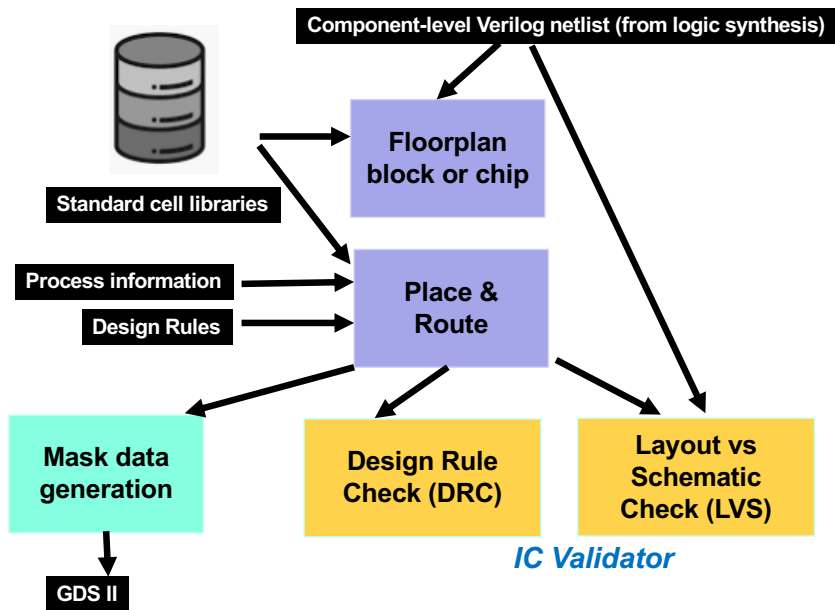
```
module lfsr4 ( clk , rst , data_out ) ;
input  clk ;
input  rst ;
output [3:0] data_out ;

\*SEQGEN.1839103_120_2 \sreg_reg[4] ( .clear ( rst ) , .preset ( 1'b0 ) ,
    .next_state ( data_out[2] ) , .clocked_on ( clk ) , .data_in ( 1'b0 ) ,
    .enable ( 1'b0 ) , .Q ( data_out[3] ) , .synch_clear ( 1'b0 ) ,
    .synch_preset ( 1'b0 ) , .synch_toggle ( 1'b0 ) , .synch_enable ( 1'b1 ) ,
    .scan_enable ( 1'b0 ) , .scan_in ( 1'b0 ) ) ;
\*SEQGEN.1839103_120_2 \sreg_reg[3] ( .clear ( rst ) , .preset ( 1'b0 ) ,
    .next_state ( data_out[1] ) , .clocked_on ( clk ) , .data_in ( 1'b0 ) ,
    .enable ( 1'b0 ) , .Q ( data_out[2] ) , .synch_clear ( 1'b0 ) ,
    .synch_preset ( 1'b0 ) , .synch_toggle ( 1'b0 ) , .synch_enable ( 1'b1 ) ,
    .scan_enable ( 1'b0 ) , .scan_in ( 1'b0 ) ) ;
\*SEQGEN.1839103_120_2 \sreg_reg[2] ( .clear ( rst ) , .preset ( 1'b0 ) ,
    .next_state ( data_out[0] ) , .clocked_on ( clk ) , .data_in ( 1'b0 ) ,
    .enable ( 1'b0 ) , .Q ( data_out[1] ) , .synch_clear ( 1'b0 ) ,
    .synch_preset ( 1'b0 ) , .synch_toggle ( 1'b0 ) , .synch_enable ( 1'b1 ) ,
    .scan_enable ( 1'b0 ) , .scan_in ( 1'b0 ) ) ;
\*SEQGEN.1839103_210_2 \sreg_reg[1] ( .clear ( 1'b0 ) , .preset ( rst ) ,
    .next_state ( N0 ) , .clocked_on ( clk ) , .data_in ( 1'b0 ) ,
    .enable ( 1'b0 ) , .Q ( data_out[0] ) , .synch_clear ( 1'b0 ) ,
    .synch_preset ( 1'b0 ) , .synch_toggle ( 1'b0 ) , .synch_enable ( 1'b1 ) ,
    .scan_enable ( 1'b0 ) , .scan_in ( 1'b0 ) ) ;
WVGTECH_XOR2 C20 ( .I0 ( data_out[2] ) , .I1 ( data_out[3] ) , .Z ( N0 ) ) ;
endmodule
```

synthesized netlist
in Verilog (before tech map)

Here is how Fusion Compiler turns the **lfsr4.sv** source code into a netlist of connected standard cells. As can be seen here, the synthesized circuit has four D-FF (two variants: SEQGEN.1839103_120_2 and SEQGEN.1839103_210_2), and an XOR gate (WVGTECH_XOR2).

FC Placement and Routing in context



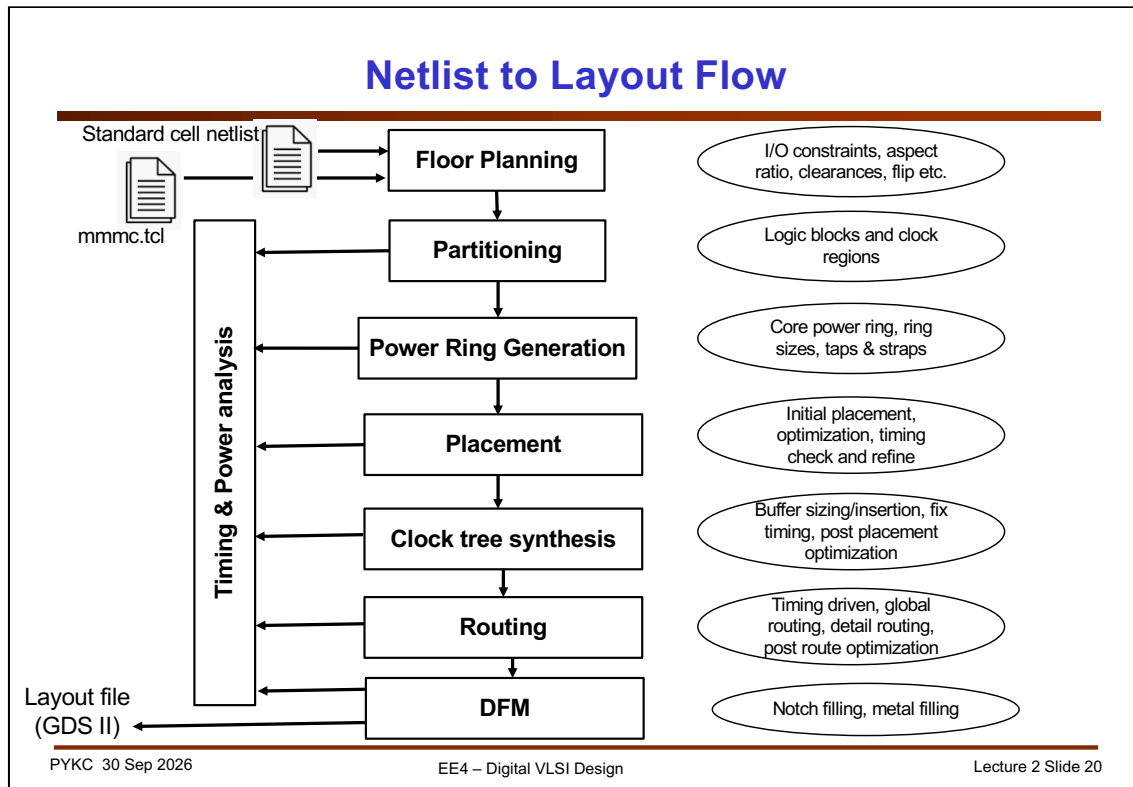
After synthesis, the tool to use to produce the actual layout is the placement and routing tools (PnR).

Fusion Compiler's PnR tool. It takes synthesized netlist and standard cell library from the PDK, perform floorplanning (rough placement with global routing) before placing the cells on the chip and conduct detail routing. Various timing-driven optimization steps are conducted before a final design is achieved.

The resulting layout is checked for design rule violation (Design Rule Checker DRC) and verified to be identical to the schematic (i.e. synthesized netlist) using LVS. The place-and-routed design must also be verified through simulation.

Fusion Compiler performs basic DRC and LVS. However, detail DRC and LVS are done through another set of tools known as IC Validator before signoff. This will be covered later in the course.

Finally, once everything are verified, the synthesized and place-and-routed design is used to generate mask data set in GDS II format, and ship to the foundry. That is known as the **tapeout process**.



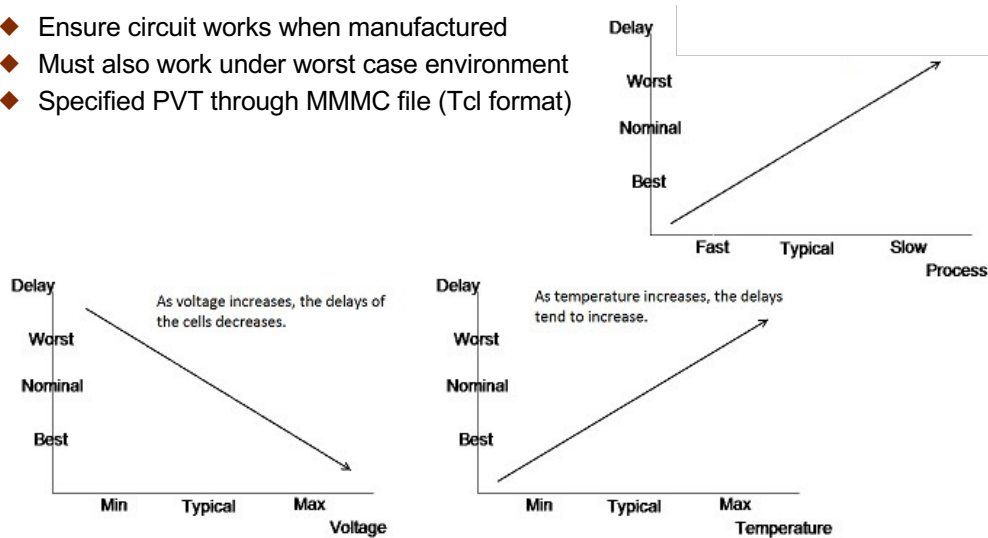
This slide shows the details steps that a typical PnR tool goes through and what each step achieves.

Note that Fusion Compiler requires additional information supplied by the file “mmmc.tcl”. This will be explained later.

Further, note that at each stage of PnR, FC performs timing analysis to modify the layout to optimize the timing – a process known as timing-driven PnR.

What is PVT?

- ◆ PVT stands for Process, Voltage and Temperature
- ◆ Need to tell Place-and-Route tools these variations
- ◆ Ensure circuit works when manufactured
- ◆ Must also work under worst case environment
- ◆ Specified PVT through MMMC file (Tcl format)



MMMC file is a specification file that inform PnR process about variations that the chip will experience. These variations include process (P), supply voltages (V) and temperature (T), collectively known as PVT.

The three graphs above depict how each of these three factors affects the gate delays. These variation information is provided in the PDK so that the PnR tool will create a layout that meets the timing specification under the worst-case condition (called corners).

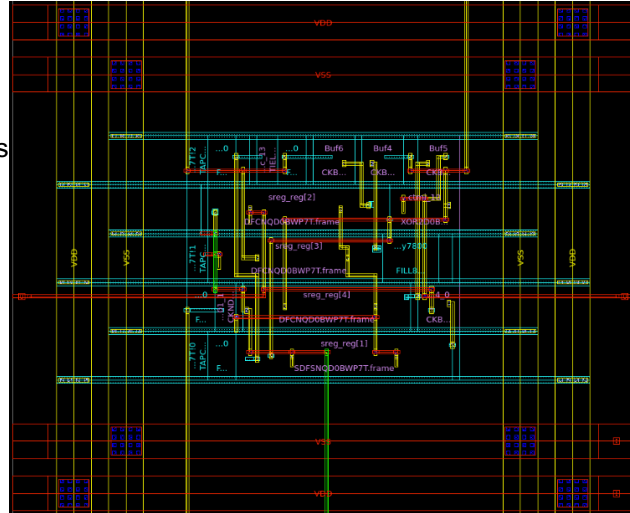
The mmmc.tcl file provides such information to Innovus. Here is the mmmc_timing.tcl script used for Lab 1. It is divided into separate sections that specify different aspects of PVT variations and their effects on the delay.

The mmmc.tcl file, together with the synthesized Verilog netlist of standard cells are the inputs to Fusion Compiler for perform PnR.

Results from PnR in Lab 1 Task 3

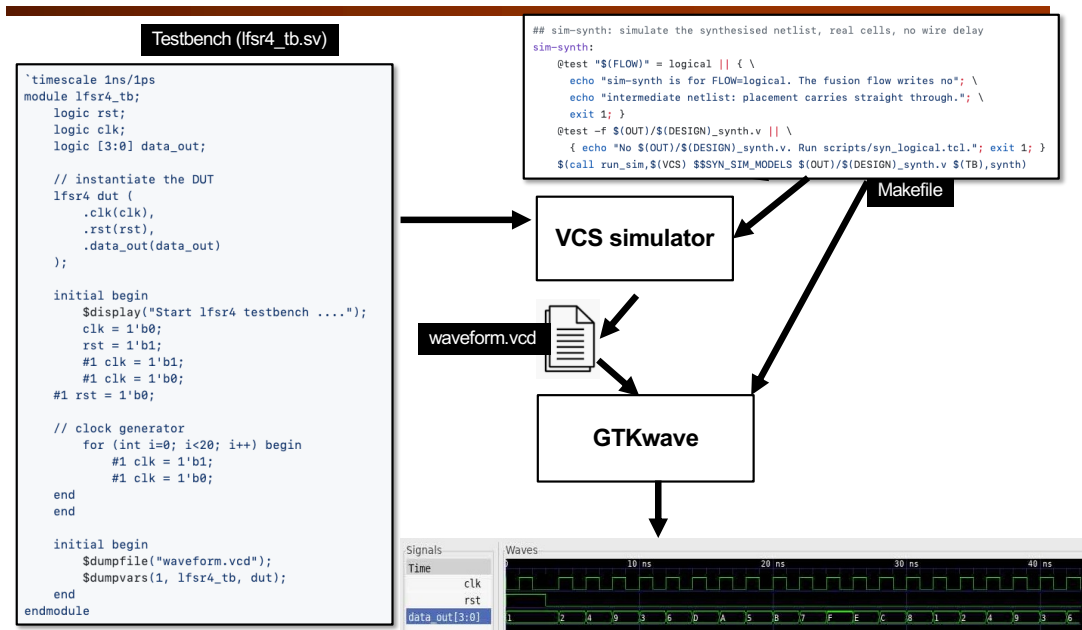
◆ To produce physical layout of the LFSR circuit, we need to:

1. Perform floorplanning
2. Perform placement
3. Synthesize the clock tree
4. Perform routing
5. Check the PnR results for errors
6. Export reports and outputs



In Lab 1, you will go through the entire physical layout generation step using Fusion Compiler. The final layout you should get is shown in the slide above.

Simulation using VCS



After completing the synthesis and PnR process, we need to verify that the finished circuit function correctly. This can be achieved through digital simulation VCS.

You can use VCS to simulate three versions of the design:

- 1) rtl – your source code;
- 2) synth – after logic synthesis in task 2
- 3) layout – final PnR netlist (without wire delay etc.) in task 3

Before simulation can be done, one has to create a testbench design for the circuit. The output is a waveform file (waveform.vcd) which can be viewed with GTKwave.

Fusion Compiler's Mega Commands



source: Fusion Compiler User Guide

Here are the main Fusion Compiler commands that perform each of the key steps in compiling RTL to final layout. Lab 1 effectively shows you how the entire flow turns a simple digital circuit into silicon layout – ALL IN ONE SINGLE LAB SESSION!

ENJOY!

Further Reading

- ◆ Publicly available standard cell library:
https://www.vlsitechnology.org/html/cells/vsclib013/lib_gif_index.html
- ◆ Synopsys Fusion Compiler User Guide:
https://spdocs.synopsys.com/dow_retrieve/latest/dg/fcolh/pdf/fcug.pdf
- ◆ Fusion Compiler Landing webpage in SolvNet:
https://spdocs.synopsys.com/dow_retrieve/latest/home_public/fc.html
- ◆ Synopsys training page in SolvNet:
<https://training.synopsys.com/learn/course/external/view/elearning/529/fusion-compiler-synthesis-and-design-implementation-jumpstart>
- ◆ “New VLSI Tools”, Gajski&Kuhn ([10.1109/MC.1983.1654264](https://doi.org/10.1109/MC.1983.1654264))

TSMC standard cell library is under NDA and therefore its details cannot be disclosed. However, list in the slide is an excellent open-source standard cell library including most of the commonly used gates and circuit snippets.

You can access all training materials and User Guides for Synopsys by creating an account on their SolvNet website.

Access to Documents – SolvNet Registration

SolvNetPlus Account Registration

- Visit SolvNetPlus:
 - <https://solvnetplus.synopsys.com/>
- Click on:
 - "Register – Create Account"
- Fill out the required information
 - University email addresses and Site ID are required
 - Site ID for **Imperial College** is siteID 2727
 - Registration may take up to 24 hours to process

The left screenshot shows the Synopsys SolvNetPlus 'Sign In' page. It has a 'Sign In' button and a 'Need trouble logging in?' link. A red arrow points from the 'Register' link in the list to the 'Register' button on this page.

The right screenshot shows the 'SolvNetPlus Self Registration' form. It includes fields for 'First Name', 'Last Name', 'Email', 'Password', 'Confirm Password', 'Phone', 'Address', 'City', 'State', 'Zip', and 'Country'. There is also a 'Site ID' field with a dropdown menu. A 'Register' button is at the bottom right.